SISO-UNAL: Sistema de Apoyo a la Inscripción de Asignaturas de la UNAL con Lista de Espera por Prioridad, Sugerencia de Grupos y Reportes de Demanda

Bryan Gustavo Llontop Angulo, Adrien Roho, Diego Fernando Castillo Albino
Universidad Nacional de Colombia
Sede Bogotá — Estructuras de Datos (DS3), 2026-2
Bogotá D.C., Colombia
{TODO-correo-bryan, TODO-correo-adrien, TODO-correo-diego}@unal.edu.co

***Resumen*—La inscripción de asignaturas en la Universidad Nacional de Colombia, sede Bogotá, involucra a cerca de 28 700 estudiantes de pregrado que inscriben entre cuatro y cinco asignaturas cada uno. El acceso al Sistema de Información Académica (SIA) se organiza mediante citas asignadas según una jerarquía de prelación institucional; sin embargo, dentro de cada franja y durante el periodo de adiciones y cancelaciones, los cupos se asignan a quien completa primero la operación, sin considerar dicha prelación. Además, cuando un grupo está lleno, el estudiante debe buscar manualmente otro grupo sin cruce de horario, y la universidad no cuenta con información sobre la demanda insatisfecha. Este proyecto propone SISO-UNAL, un sistema en Java que aborda estos tres problemas: (i) una *lista de espera por prioridad* por grupo que reasigna automáticamente cada cupo liberado al estudiante de mayor prioridad oficial que haya aceptado esa asignación, a diferencia de los sobrecupos, que crean cupos adicionales; (ii) la sugerencia de grupos alternativos sin cruce de horario; y (iii) reportes de demanda insatisfecha. La primera entrega implementa el núcleo del sistema, la lista de espera, con estructuras implementadas desde cero: una cola de prioridad sobre una lista enlazada ordenada, una lista enlazada para los inscritos, una cola FIFO para el orden de atención de una cita y una pila para deshacer acciones en el carrito de selección. La cola de prioridad se define mediante una interfaz, de modo que en las siguientes entregas pueda reemplazarse por un montículo (*heap*). El prototipo actual, ejecutable por consola, demuestra la clasificación por grupos de prelación, el ordenamiento por prioridad, la lista de espera y la reasignación automática de un cupo liberado. La sugerencia de grupos y los reportes se desarrollarán en las entregas 2 y 3.**

***Index Terms*—estructuras de datos, colas de prioridad, listas enlazadas, asignación de cupos, inscripción académica, análisis de complejidad**

I. Introducción y planteamiento del problema

I-A. Contexto

En la sede Bogotá de la Universidad Nacional de Colombia (UNAL) hay alrededor de 32 090 estudiantes matriculados, de los cuales aproximadamente 28 700 son de pregrado [1]. Cada semestre, estos estudiantes inscriben sus asignaturas a través del SIA. Para distribuir la carga sobre la plataforma, la universidad no abre la inscripción a todos al mismo tiempo: asigna a cada estudiante una *cita*, es decir, una franja de 30 minutos dentro de una ventana de tres días, a partir de la cual puede inscribir.

A partir del periodo 2026-2S, el orden de las citas se rige por la Circular 13 de 2026 de la Vicerrectoría Académica [3], que define cuatro grupos de prelación excluyentes: (1) estudiantes en situación de discapacidad; (2) estudiantes en vulnerabilidad académica o riesgo de abandono (por ejemplo, PAPA menor o igual a 3.1, o programas especiales como PEAMA); (3) estudiantes con avance curricular igual o superior al 80 %, ordenados por avance; y (4) los demás estudiantes, ordenados mediante el Índice de Prioridad de Asignación (IPA).

El calendario de inscripción tiene cinco fases [2]: (1) consulta de la citación, que asigna a cada estudiante un día y una hora según su grupo de prelación; (2) primera cita de inscripción en el SIA, en los días previos al inicio de clases; (3) adiciones y cancelaciones libres durante las semanas 1 y 2 del semestre, con una segunda franja de acceso general y sin pérdida de créditos; (4) cancelación ordinaria entre las semanas 3 y 8, en la que solo se puede cancelar y los créditos se pierden; y (5) un periodo extemporáneo, desde la semana 9, en el que la cancelación requiere una solicitud aprobada por el Comité Asesor de Carrera y el Consejo de Facultad.

La Tabla I muestra la distribución aproximada de los estudiantes de pregrado por facultad. La Facultad de Ingeniería concentra cerca del 29 % del pregrado y, además, varias asignaturas (cálculo, inglés, asignaturas de libre elección) son compartidas entre facultades, lo que genera una alta competencia por los mismos grupos.

I-B. Situación que se desea resolver

El sistema de citas resuelve *quién entra primero*, pero deja sin resolver los siguientes problemas:

Tabla I
Distribución aproximada del pregrado por facultad, sede Bogotá

Facultad	Estudiantes	%
Ingeniería	8 320	29.0
Ciencias Humanas	4 880	17.0
Ciencias	3 440	12.0
Medicina	2 580	9.0
Artes	2 300	8.0
Ciencias Económicas	2 010	7.0
Derecho, C. Políticas y Sociales	1 720	6.0
Ciencias Agrarias	1 000	3.5
Medicina Veterinaria y Zootecnia	860	3.0
Enfermería	860	3.0
Odontología	730	2.5
Total	**28 700**	**100.0**

1. **Cupos liberados sin prioridad.** Durante el periodo de adiciones y cancelaciones (fase 3, semanas 1 y 2 del semestre), cuando un estudiante cancela, el cupo liberado lo toma quien primero recarga el SIA. Lo mismo ocurre dentro de cada franja, compartida por cientos de estudiantes. No existe una lista de espera que asigne los cupos según la prioridad institucional.
2. **Grupos llenos sin alternativas.** Si el grupo deseado está lleno, el estudiante debe buscar manualmente otro grupo de la misma asignatura que tenga cupo y no le genere cruce de horario con lo que ya inscribió.
3. **Demanda insatisfecha desconocida.** No se registra cuántos estudiantes quedaron sin cupo en cada asignatura ni cuáles asignaturas tienen más estudiantes en espera, información útil para planear la oferta de semestres futuros. Además, en el periodo de cancelación ordinaria (fase 4, semanas 3 a 8), en el que ya no se permiten inscripciones, los cupos liberados se pierden sin que quede registro.

I-C. Relevancia

El problema afecta a toda la población de pregrado en cada periodo académico. Que un cupo dependa de la velocidad de conexión o de recargar la página en el momento exacto contradice el objetivo de equidad de la propia Circular 13 [3]. Además, las asignaturas obligatorias de los primeros semestres y las compartidas entre facultades son precisamente las que determinan el avance de los estudiantes.

I-D. Dificultades computacionales

- **Escala.** Entre 115 000 y 143 000 solicitudes de inscripción ($28\,700 \times 4$–5) en pocos días. Una operación $O(n)$ repetida por cada solicitud puede volverse costosa.
- **Orden total de prioridad.** Hay que ordenar a 28 700 estudiantes según un criterio compuesto (grupo de prelación y puntaje dentro del grupo). Con una lista ordenada esto cuesta $O(n^2)$ en el peor caso, del orden de 8.2×10^8 comparaciones; con un montículo, $O(n \log n)$, del orden de 4.2×10^5.
- **Extracción repetida del máximo.** Cada cupo liberado exige obtener al estudiante de mayor prioridad en espera; la estructura debe soportar inserción y extracción eficientes.
- **Consistencia.** Un estudiante puede estar en la lista de espera de varios grupos de la misma asignatura; al obtener uno, debe salir de los demás.
- **Cruces de horario.** Sugerir un grupo alternativo exige comparar su horario con todos los grupos ya inscritos por el estudiante.

I-E. Propuesta general

SISO-UNAL no reemplaza el sistema de citas: lo simula y lo complementa con tres funciones: (i) una lista de espera por prioridad por grupo, que reasigna automáticamente cada cupo liberado al estudiante de mayor prioridad oficial, siempre que este haya aceptado previamente la asignación automática; (ii) la sugerencia de grupos alternativos sin cruce de horario cuando el grupo deseado está lleno; y (iii) reportes de demanda insatisfecha y de asignaturas con más estudiantes en espera. La primera entrega implementa la función (i), que es el núcleo del sistema; las funciones (ii) y (iii) se desarrollan en las entregas 2 y 3.

I-F. Diferencia con los sobrecupos

La UNAL contempla *sobrecupos*: autorizaciones para inscribir a un estudiante por encima del cupo de un grupo. Se solicitan durante la inscripción y las adiciones, cuando el cupo regular del grupo aparece en cero. El trámite depende de cada facultad: un módulo de solicitudes en el SIA, formularios internos o un correo directo al docente. El docente o el Comité Asesor de Carrera evalúan la capacidad física del salón y la justificación del estudiante, con preferencia para quienes necesitan la asignatura para graduarse o para no retrasar su plan de estudios. Si se aprueba, la secretaría académica inscribe la asignatura manualmente o habilita un cupo exclusivo para el estudiante, quien en todo caso debe tener créditos disponibles.

Los sobrecupos son un mecanismo complementario que no afecta directamente al proyecto: SISO-UNAL trabaja únicamente sobre los cupos regulares de cada grupo, y un sobrecupo aprobado simplemente aumenta el cupo de ese grupo sin cambiar el funcionamiento de la lista de espera. Por eso no se modelan. Aun así, los sobrecupos atenúan el problema solo parcialmente. Crean cupos adicionales limitados por el salón y la carga docente; el trámite es manual, varía entre facultades y se resuelve caso por caso; y sus criterios no siguen necesariamente el orden de prelación de la Circular 13. Sobre todo, no resuelven qué pasa con los cupos regulares que se liberan, que siguen yendo a quien recargue primero el SIA. La lista de espera es complementaria: no crea cupos, sino que reasigna de forma automática y por prioridad los cupos existentes que se liberan. Además, puede apoyar el trámite de sobrecupos: ofrece un criterio objetivo y uniforme entre facultades para

decidir a quién otorgarlos (el primero en espera) y datos sobre cuántos estudiantes esperan cupo en cada grupo.

II. Objetivos

II-A. Objetivo general

Diseñar, implementar y evaluar en Java un sistema que simule la inscripción de asignaturas por citas de la UNAL y que la complemente con una lista de espera por prioridad, la sugerencia de grupos sin cruce de horario y reportes de demanda, utilizando estructuras de datos implementadas por el equipo y analizando la complejidad de sus operaciones.

II-B. Objetivos específicos

1. Clasificar a cada estudiante en uno de los cuatro grupos de prelación de la Circular 13 y definir un orden total de prioridad entre estudiantes.
2. Mantener, para cada grupo lleno, una lista de espera por prioridad con los estudiantes que aceptaron la asignación automática, y reasignar cada cupo liberado al estudiante de mayor prioridad.
3. Simular la atención de las citas en orden de prioridad y permitir que el estudiante deshaga la última acción de su carrito de selección.
4. Sugerir, cuando el grupo deseado esté lleno, grupos de la misma asignatura con cupo y sin cruce de horario.
5. Generar reportes de demanda insatisfecha por asignatura y de cupos perdidos en el periodo de cancelación ordinaria.
6. Analizar teórica y experimentalmente el costo de las operaciones principales y comparar implementaciones alternativas (lista ordenada frente a montículo; búsqueda lineal frente a tabla hash).
7. Comparar, mediante simulación, la asignación actual ("gana el más rápido") con la asignación por lista de espera con prioridad, midiendo cuántas veces un estudiante de menor prioridad obtiene un cupo antes que uno de mayor prioridad.

Los objetivos 1 a 3 se abordan en la primera entrega; los objetivos 4 a 7, en las entregas 2 y 3.

III. Alcance y limitaciones

III-A. Funcionalidades dentro del alcance (semestre)

- Registro de grupos de asignaturas (cupo y horario) y de estudiantes de pregrado.
- Clasificación en grupos de prelación y orden total de prioridad.
- Simulación de las citas: franjas de atención y procesamiento FIFO de solicitudes.
- Carrito de selección con operación de deshacer.
- Lista de espera por prioridad y reasignación automática de cupos liberados.
- Fases del calendario 2 a 4: primera cita, adiciones y cancelaciones (semanas 1 y 2) y cancelación ordinaria (semanas 3 a 8).
- Detección de cruces de horario y sugerencia de grupos alternativos.
- Validación de prerrequisitos.
- Reportes de demanda insatisfecha y de cupos perdidos.
- Generación de datos sintéticos a escala real (28 700 estudiantes) y evaluación experimental.

III-B. Fuera del alcance

- Integración con el SIA real o con bases de datos institucionales.
- El trámite de sobrecupos; el sistema solo reasigna cupos existentes.
- La fase 1 (consulta de la citación) como trámite; el sistema solo usa su resultado, el orden de prioridad.
- El periodo extemporáneo (fase 5), que depende de decisiones de comités.
- La bolsa de créditos y la validación de la carga mínima de 10 créditos al final de la fase 3.
- Estudiantes de posgrado (alrededor de 3 390).
- Interfaz gráfica o web; el sistema se ejecuta por consola.
- Concurrencia real y carga sobre servidores; la competencia dentro de una franja se simula.
- El detalle de los ajustes razonables del Grupo 1 y de cada programa especial del Grupo 2 (se representan con indicadores booleanos), y los criterios propios de cada facultad para estudiantes de primera matrícula.

III-C. Restricciones técnicas

- Java 17 o superior, sin dependencias externas.
- Las estructuras de datos del proyecto se implementan desde cero; no se utilizan las colecciones de `java.util` (por ejemplo, `ArrayList`, `LinkedList` o `PriorityQueue`).
- El código y el documento se mantienen en el repositorio oficial del curso.

III-D. Supuestos

- El IPA es un dato de entrada ya calculado por la universidad; el sistema no lo recalcula. La circular no publica los pesos de sus factores.
- El grupo de prelación se deriva de atributos del estudiante: discapacidad acreditada (Grupo 1); PAPA ≤ 3.1 o indicador de vulnerabilidad académica (Grupo 2); avance ≥ 80 % (Grupo 3); en otro caso, Grupo 4.
- Dentro de cada grupo se ordena así: Grupo 1 por PAPA descendente; Grupo 2 por PAPA ascendente (mayor riesgo primero); Grupo 3 por avance descendente y luego PAPA descendente; Grupo 4 por IPA descendente. El último desempate es el código del estudiante, para obtener un orden total y reproducible.
- El estudiante da su aceptación de la asignación automática al solicitar un grupo lleno, y puede retirarla saliendo de la lista. Cuando se libera un cupo, la asignación es inmediata. Una oferta con plazo para

Tabla II
Requerimientos funcionales y entrega en la que se implementan

ID	Descripción	Entrega
RF-01	Registrar grupos de asignaturas con cupo.	1
RF-02	Registrar estudiantes y calcular su grupo de prelación.	1
RF-03	Ordenar a los estudiantes por prioridad oficial.	1
RF-04	Atender en orden FIFO a los estudiantes de una cita.	1
RF-05	Inscribir a un estudiante si hay cupo.	1
RF-06	Si el grupo está lleno y el estudiante acepta la asignación automática, insertarlo en la lista de espera por prioridad; permitirle retirarse de ella.	1
RF-07	Cancelar una inscripción y reasignar el cupo al primero de la lista de espera.	1
RF-08	Agregar grupos al carrito y deshacer la última acción.	1
RF-09	Mostrar inscritos y lista de espera de un grupo.	1
RF-10	Repartir a los estudiantes en franjas de citas.	2
RF-11	Al reasignar, retirar al estudiante de las demás listas de la misma asignatura.	2
RF-12	Buscar estudiantes y grupos por código en tiempo esperado $O(1)$.	2
RF-13	Simular las fases del calendario académico.	2
RF-14	Detectar cruces de horario.	2
RF-15	Generar datos sintéticos a escala real.	2
RF-16	Validar prerrequisitos de una asignatura.	3
RF-17	Sugerir grupos alternativos sin cruce de horario.	3
RF-18	Reportar demanda insatisfecha y cupos perdidos.	3

aceptar o rechazar el cupo se modelará en la segunda entrega, junto con las fases del calendario.

- El orden de atención de una cita coincide con el orden de prioridad; desde la segunda entrega se simula una llegada aleatoria dentro de cada franja para modelar la competencia real.
- Cada estudiante inscribe entre cuatro y cinco asignaturas.

III-E. Límites razonables

Con un equipo de tres personas y tres entregas, el proyecto se limita a la simulación en memoria de un periodo de inscripción de la sede Bogotá. La fidelidad normativa se prioriza en la regla de prioridad, que es la que afecta directamente a las estructuras de datos; los demás detalles administrativos se simplifican y se documentan como supuestos.

IV. Requerimientos principales

IV-A. Requerimientos funcionales

La Tabla II lista los requerimientos funcionales. Cada uno corresponde a una operación concreta que puede ejecutarse y verificarse en el programa.

IV-B. Requerimientos no funcionales

- **RNF-01 (Eficiencia).** Las operaciones de cola y pila deben ser $O(1)$; la extracción del estudiante de mayor prioridad debe ser $O(1)$ con la lista ordenada y $O(\log n)$ con el montículo.
- **RNF-02 (Escalabilidad).** La simulación completa debe poder ejecutarse con 28 700 estudiantes desde la segunda entrega.
- **RNF-03 (Reproducibilidad).** Los experimentos usarán semillas fijas para que cualquier persona obtenga los mismos resultados.
- **RNF-04 (Mantenibilidad).** Las estructuras se exponen mediante interfaces, de modo que una implementación pueda reemplazarse sin modificar el resto del sistema.

V. Trabajo relacionado inicial

Sistemas institucionales. El SIA de la UNAL gestiona la inscripción mediante citas cuyo orden fija la Circular 13 de 2026 [3]. Este esquema garantiza el acceso por prelación, pero, una vez abierta la franja, la asignación de cupos opera como "primero en llegar, primero en ser atendido" y no existe una reserva de cupos ni una lista de espera para los cupos que se liberan.

Mecanismos de asignación de cursos. La literatura de diseño de mecanismos estudia la asignación de cupos como un problema de asignación multiunidad. El mecanismo de *dictadura serial*, en el que los agentes eligen uno tras otro según un orden de prioridad, ha sido analizado formalmente [4]; el sistema de citas de la UNAL puede verse como una variante de este esquema ordenada por la Circular 13. Budish y Cantillon [5] estudiaron el mecanismo de asignación de cursos de la Escuela de Negocios de Harvard y mostraron cómo los estudiantes responden estratégicamente a sus reglas. Budish [6] propuso un mecanismo basado en equilibrio competitivo aproximado, implementado posteriormente a gran escala en la Escuela Wharton bajo el nombre *Course Match* [7]. A diferencia de esos trabajos, este proyecto no rediseña el mecanismo principal: conserva las citas y se concentra en la reasignación de los cupos que se liberan, donde hoy no se aplica ninguna prioridad.

Estructuras de datos. La cola de prioridad es el tipo abstracto natural para obtener repetidamente el elemento de mayor prioridad. Sus implementaciones clásicas (lista no ordenada, lista ordenada y montículo binario) y sus costos se presentan en [8]–[10]; el montículo permite inserción y extracción en $O(\log n)$ y, aplicado a ordenamiento, el algoritmo *heapsort* en $O(n \log n)$ [10]. Las colas FIFO son el modelo habitual para atender solicitudes en orden de llegada, y las pilas, para implementar mecanismos de deshacer en editores y aplicaciones [8].

VI. Diseño de la solución

VI-A. Arquitectura

El sistema se organiza en tres capas (Fig. 1):

- **Estructuras de datos:** `Node`, `LinkedList`, `LinkedQueue`, `LinkedStack`, la interfaz `PriorityQueueADT` y su implementación

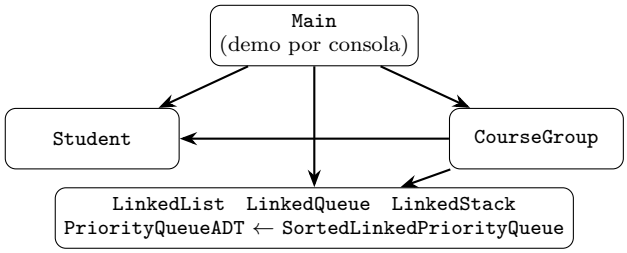


Figura 1. Arquitectura en capas de SISO-UNAL (Entrega 1). Las flechas indican dependencia.

`SortedLinkedPriorityQueue`. Son genéricas y no conocen el dominio.

- **Modelo:** `Student` (atributos, grupo de prelación y orden de prioridad mediante `Comparable`) y `CourseGroup` (cupo, inscritos, lista de espera, solicitud y cancelación).
- **Presentación:** `Main` ejecuta una demostración guiada por consola. En la segunda entrega se agregará una clase de servicio que coordine el catálogo, las franjas de citas y las fases del calendario.

VI-B. Modelo de datos

- `Student`: código, nombre, discapacidad (booleano), PAPA, vulnerabilidad académica (booleano), avance curricular (%) e IPA. Implementa `Comparable<Student>`: `a.compareTo(b) <0` significa que `a` tiene *mayor* prioridad que `b`. La comparación cuesta $O(1)$. La igualdad se define por el código.
- `CourseGroup`: código y nombre de la asignatura, número de grupo, cupo, lista de inscritos (`LinkedList<Student>`) y lista de espera (`PriorityQueueADT<Student>`, instanciada como `SortedLinkedPriorityQueue`). Ofrece `request` (inscribir o poner en espera si el estudiante acepta la asignación automática; el resultado es un `RequestResult`), `leaveWaitlist` (retirarse de la lista) y `cancel` (cancelar y reasignar).

VI-C. Flujo de información

1. Se crean los grupos y los estudiantes; cada estudiante calcula su grupo de prelación a partir de sus atributos.
2. Los estudiantes se ordenan insertándolos en una cola de prioridad y extrayéndolos uno a uno; en ese orden se encolan en la cola FIFO de la cita.
3. Se desencolan los estudiantes y cada uno solicita un grupo: queda inscrito si hay cupo; si no, entra a la lista de espera solo si acepta la asignación automática.
4. Durante las adiciones, nuevas solicitudes sobre un grupo lleno entran a la lista de espera en la posición que les corresponde por prioridad, sin importar el momento de llegada.
5. Cada cancelación libera un cupo que se asigna al primero de la lista de espera.

Algoritmo 1 Orden de atención de una cita

Entrada: lista de estudiantes E ($|E| = n$)
Salida: cola FIFO C con los estudiantes en orden de prioridad

```
P ← nueva cola de prioridad vacía
for cada e ∈ E do
    P.INSERT(e)                          ▷ O(n) con lista ordenada
end for
C ← nueva cola FIFO vacía
while no P.ISEMPTY() do
    C.ENQUEUE(P.REMOVEHIGHEST())         ▷ O(1)
end while
return C
```

Algoritmo 2 Solicitud de inscripción (`request`)

Entrada: estudiante e, grupo g con cupo c, aceptación a

```
if e ∈ g.inscritos o e ∈ g.espera then
    error: solicitud duplicada           ▷ O(c + w)
end if
if |g.inscritos| < c then
    g.inscritos.ADDLAST(e)               ▷ O(1)
    return INSCRITO
end if
if no a then
    return RECHAZADO                     ▷ no aceptó la asignación
end if
g.espera.INSERT(e)                       ▷ O(w)
return ENESPERA
```

6. El carrito de selección (pila) permite deshacer la última acción.

VI-D. Algoritmos principales

El Algoritmo 1 ordena a los estudiantes y arma la cola de la cita; el Algoritmo 2 procesa una solicitud y el Algoritmo 3 reasigna un cupo liberado.

VI-E. Ejemplo de funcionamiento

El prototipo ejecuta el siguiente escenario sobre el grupo 1 de Cálculo Diferencial, con cupo 2, y los estudiantes de la Tabla III.

1. *Cita:* la cola de prioridad ordena a Ana, Carla, Elena y Felipe, que son atendidos en ese orden. Ana y Carla quedan inscritas. Elena acepta la asignación automática y entra a la lista de espera; Felipe no la acepta y no entra: [Elena].
2. *Adiciones:* Daniel solicita el grupo más tarde y acepta la asignación automática. Aunque llegó después, entra a la lista por delante de Elena por su prioridad: [Daniel, Elena].
3. *Cancelación:* Carla cancela. El cupo se asigna automáticamente a Daniel. En el sistema actual, el cupo lo habría obtenido quien recargara primero el SIA.
4. *Carrito:* Felipe agrega dos grupos a su carrito y deshace el último.

Algoritmo 3 Cancelación y reasignación automática (`cancel`)

Entrada: estudiante e inscrito en el grupo g

1: $g.inscritos$.REMOVE(e) ▷ $O(c)$
2: **if** $g.espera$.ISEMPTY() **then**
3: **return nulo**
4: **end if**
5: $s \leftarrow g.espera$.REMOVEHIGHEST() ▷ $O(1)$
6: $g.inscritos$.ADDLAST(s) ▷ $O(1)$
7: **return** s

Tabla III
ESTUDIANTES DEL EJEMPLO (DATOS ILUSTRATIVOS)

Estudiante	Atributo decisivo	Grupo	Orden
Ana	discapacidad	1	1.º
Carla	avance 92 %	3	2.º
Daniel	avance 85 %	3	3.º
Elena	IPA 7.5	4	4.º
Felipe	IPA 6.0	4	5.º

VII. Estructuras de datos seleccionadas

Cada estructura se incorpora porque responde a una necesidad concreta del problema. La Tabla IV resume la selección; a continuación se justifica cada una. Se usa n para el número de estudiantes, c para el cupo de un grupo y w para el tamaño de una lista de espera.

VII-A. Cola de prioridad (lista de espera)

Es la estructura central del proyecto. Debe permitir insertar a un estudiante en cualquier momento y extraer siempre al de mayor prioridad. En esta entrega se implementa con una lista enlazada ordenada: la inserción recorre la lista hasta encontrar su posición ($O(w)$) y la extracción toma el primer nodo ($O(1)$). Ante prioridades iguales, el nuevo elemento queda después de los existentes, aunque en la práctica no hay empates porque el orden de `Student` es total. Se eligió esta versión porque usa únicamente estructuras ya estudiadas en el curso y porque, en una lista de espera, las extracciones (cada cupo liberado) deben ser inmediatas. La interfaz `PriorityQueueADT` permitirá sustituirla por un montículo binario, con inserción y extracción en $O(\log w)$, sin modificar `CourseGroup`. Ambas implementaciones se compararán experimentalmente.

La misma cola de prioridad se usa para ordenar a los estudiantes antes de la cita (Algoritmo 1). Con la lista ordenada, este ordenamiento cuesta $O(n^2)$ en el peor caso; con el montículo será $O(n \log n)$, equivalente a *heapsort*.

VII-B. Lista enlazada (inscritos)

Representa los inscritos de un grupo, cuyo número cambia con cada inscripción y cancelación. Mantener una referencia al último nodo permite `addLast` en $O(1)$. Verificar si un estudiante ya está inscrito y eliminarlo al cancelar cuestan $O(c)$; como c es el cupo de un grupo (decenas de estudiantes), este costo es aceptable.

VII-C. Cola FIFO (atención de la cita)

Dentro de una cita, el SIA atiende a los estudiantes según el orden en que completan sus operaciones. La cola FIFO representa exactamente ese comportamiento, con `enqueue` y `dequeue` en $O(1)$. Se eligió la versión enlazada porque el número de estudiantes por cita varía y no exige fijar una capacidad.

VII-D. Pila (carrito y deshacer)

Antes de confirmar, el estudiante agrega grupos a su carrito y puede deshacer la última acción. Este comportamiento es LIFO: la acción que se deshace siempre es la más reciente, por lo que `push` y `pop` en $O(1)$ son suficientes.

VII-E. Resumen de complejidad de las operaciones del sistema

- Ordenar a los estudiantes para la cita: $O(n^2)$ (lista ordenada) $\rightarrow O(n \log n)$ (montículo).
- Solicitud de inscripción (`request`): $O(c + w)$, dominado por la verificación de duplicados; la inscripción en sí es $O(1)$ y la entrada a la lista de espera, $O(w)$.
- Cancelación con reasignación (`cancel`): $O(c) + O(1)$.
- Deshacer en el carrito: $O(1)$.

VIII. Estado de la implementación

VIII-A. Funcionalidades implementadas

- Estructuras genéricas propias: `Node`, `LinkedList`, `LinkedQueue`, `LinkedStack`, `PriorityQueueADT` y `SortedLinkedPriorityQueue`. Las operaciones sobre estructuras vacías lanzan una excepción con un mensaje claro.
- `Student`: grupo de prelación y orden total de prioridad (`compareTo`), con el IPA como dato de entrada.
- `CourseGroup`: inscripción con cupo, lista de espera por prioridad, aceptación de la asignación automática y retiro de la lista, rechazo de solicitudes duplicadas y cancelación con reasignación automática del cupo (RF-01 a RF-07 y RF-09). El resultado de cada solicitud se representa con el enumerado `RequestResult` (inscrito, en espera o rechazado).
- Carrito con deshacer (RF-08).
- Demostración guiada por consola (`Main`) con el escenario del ejemplo de la sección de diseño.

VIII-B. Funcionalidades parcialmente implementadas

- Simulación de citas: se atiende una cita en orden de prioridad; el reparto de los estudiantes en varias franjas (RF-10) se agregará en la segunda entrega.
- Orden dentro del Grupo 4: se usa solo el IPA; la subdivisión por número de matrícula se agregará en la segunda entrega.
- Interacción por consola: demostración guiada, sin menú interactivo.

Tabla IV
Estructuras de datos de la primera entrega

Estructura	Información que representa	Operaciones	Justificación	Alternativas	Costos
Cola de prioridad sobre lista ordenada (`SortedLinkedPriorityQueue`)	Lista de espera de un grupo; orden de atención de la cita	`insert`, `removeHighest`, `peekHighest`, `remove`, `contains`	El primero de la lista siempre es el de mayor prioridad; la extracción es inmediata	Lista no ordenada (inserción $O(1)$, extracción $O(n)$); montículo (entrega 2)	`insert` $O(w)$; `removeHighest` $O(1)$; `remove`, `contains` $O(w)$
Lista enlazada simple con cola (`LinkedList`)	Inscritos de un grupo	`addLast`, `remove`, `contains`	Tamaño variable; inserción al final constante gracias a la referencia al último nodo	Arreglo dinámico (búsqueda igual, inserción amortizada)	`addLast` $O(1)$; `remove`, `contains` $O(c)$
Cola FIFO enlazada (`LinkedQueue`)	Estudiantes de una cita en orden de atención	`enqueue`, `dequeue`, `peek`	Modela la atención en orden de llegada dentro de la cita	Cola circular sobre arreglo (requiere capacidad o redimensionar)	Todas $O(1)$
Pila enlazada (`LinkedStack`)	Carrito de selección del estudiante	`push`, `pop`, `peek`	Deshacer la última acción es un comportamiento LIFO	Arreglo con índice (capacidad fija)	Todas $O(1)$

VIII-C. Funcionalidades pendientes

- **Entrega 2:** oferta del cupo liberado con plazo para aceptar o rechazar; franjas de citas; retiro del estudiante de las demás listas de espera de la misma asignatura (RF-11); cola de prioridad con montículo; tabla hash para búsquedas por código; generador de datos sintéticos (28 700 estudiantes según la Tabla I); llegada aleatoria dentro de cada franja; fases del calendario; detección de cruces de horario; menú interactivo; diseño experimental.
- **Entrega 3:** grafo de prerrequisitos (recorridos BFS/DFS); sugerencia de grupos alternativos sin cruce; reportes de demanda insatisfecha y cupos perdidos; experimentos comparativos e informe final.

VIII-D. Repositorio

El código se encuentra en el repositorio oficial del curso, en `DS3/teams/team-07/projects/entrega-01/`. Se compila y ejecuta desde esa carpeta con:

```
javac -encoding UTF-8 -d out src/*.java
java -cp out Main
```

El archivo `README.md` de esa carpeta incluye la salida esperada completa del programa.

IX. Resultados y discusión

Esta sección se desarrollará en las entregas 2 y 3. Se medirán el tiempo de ordenamiento y de las operaciones de la lista de espera para $n \in \{1\,000;\ 10\,000;\ 28\,700\}$, comparando la lista ordenada con el montículo, y se cuantificará la equidad de la asignación (número de inversiones de prioridad) en el escenario actual frente al propuesto, con énfasis en la Facultad de Ingeniería y en las asignaturas compartidas.

X. Conclusiones preliminares

El problema seleccionado exige mantener un orden de prioridad sobre decenas de miles de estudiantes y extraer repetidamente el de mayor prioridad, lo que conecta directamente con los contenidos del curso. La primera entrega implementa el núcleo del sistema, la lista de espera por prioridad, con un diseño basado en interfaces que permite evolucionar hacia montículos, tablas hash y grafos sin rehacer el sistema.

Declaración de uso de inteligencia artificial

Se utilizó un asistente de inteligencia artificial (Claude, de Anthropic) para la lluvia de ideas, la organización de este documento, la revisión del código y la implementación de la lista de espera por prioridad. El contenido y el código fueron verificados, modificados e integrados por los autores, quienes asumen la responsabilidad total del resultado.

Referencias

[1] Universidad Nacional de Colombia, Dirección de Planeación y Estadística, Sede Bogotá, "Sede en cifras: Matriculados," reporte interactivo. [En línea]. Disponible en: https://planeacion.bogota.unal.edu.co/sede_en_cifras/reportes_interactivos/Matriculados/

[2] Universidad Nacional de Colombia, Dirección Nacional de Información Académica (DNINFOA), "Inscripción de asignaturas," infografía. [En línea]. Disponible en: https://dninfoa.unal.edu.co/docs/Infografia_inscripcion_asignaturas.pdf

[3] Universidad Nacional de Colombia, Vicerrectoría Académica, "Circular 13 de 2026," Bogotá, Colombia, 2026.

[4] A. Abdulkadiroğlu y T. Sönmez, "Random serial dictatorship and the core from random endowments in house allocation problems," *Econometrica*, vol. 66, no. 3, pp. 689–701, 1998.

[5] E. Budish y E. Cantillon, "The multi-unit assignment problem: Theory and evidence from course allocation at Harvard," *American Economic Review*, vol. 102, no. 5, pp. 2237–2271, 2012.

[6] E. Budish, "The combinatorial assignment problem: Approximate competitive equilibrium from equal incomes," *Journal of Political Economy*, vol. 119, no. 6, pp. 1061–1103, 2011.

[7] E. Budish, G. P. Cachon, J. B. Kessler y A. Othman, "Course Match: A large-scale implementation of approximate competitive equilibrium from equal incomes for combinatorial allocation," *Operations Research*, vol. 65, no. 2, pp. 314–336, 2017.

[8] M. T. Goodrich, R. Tamassia y M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Hoboken, NJ, USA: Wiley, 2014.
[9] R. Sedgewick y K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.
[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.